API for Data Dissemination Protocols - Evaluation with AutoCast

Sebastian Ebers*, Mohamed A. Hail†, Stefan Fischer* and Horst Hellbrück†
*Institute of Telematics, University of Lübeck, Germany
{ebers, fischer}@itm.uni-luebeck.de
†Department of Electrical Engineering and Computer Science, Lübeck University of Applied Sciences, Germany
{hail, hellbrueck}@fh-luebeck.de

***Abstract*—In the past various protocols inspired by nature and biology have been proposed to disseminate or transfer data in mobile or static ad-hoc networks. Many of them are designed for usage in wireless sensor networks or vehicular ad-hoc networks. Recently, we have developed and designed a general purpose data dissemination protocol called AutoCast in this field that we evaluated in detail by simulations. When we started to use AutoCast in real applications, we found out that the description of AutoCast is incomplete, as we provided the algorithms of AutoCast in details but did neither provide nor describe a suitable Application Programming Interface (API) and AutoCast was closely coupled to the application. The focus of this article is twofold. First, we propose an appropriate API to encapsulate data dissemination protocols like AutoCast and we specify the service interface of AutoCast in detail. This API can serve as a reference model for other nature and biologically inspired data dissemination approaches and applications. Second, we evaluate two applications based on our API with AutoCast in the field of wireless sensor networks and vehicular ad-hoc networks to illustrate the usage of the API and demonstrate the flexibility of this approach.**

***Keywords*-API, data dissemination, vehicular ad-hoc networks, wireless sensor networks, communication protocols**

I. Introduction

In recent years more and more devices are equipped with wireless interfaces. Many of these interfaces support ad-hoc mode (sometimes called peer-to-peer mode) where end devices can communicate directly without infrastructure. New standards like IEEE 802.15.4 or IEEE 802.11p are targeted for this ad-hoc mode. IEEE 802.11p is the nominated standard for vehicular ad-hoc networks (VANETs) [1], [2].

In the past various protocols inspired by nature and biology have been proposed to disseminate data in mobile or static ad-hoc networks first in the field of WSNs and VANETs. Routing protocols developed for the Internet and ad-hoc networks do not work as designed in these applications. For many applications unicast or multicast operation is not appropriate [3]. In WSNs based on IEEE 802.15.4, data sensing is performed by several nodes in an area as single sensor nodes tend to fail. Consequently, individual nodes are not important and approaches inspired by nature and biology like dissemination promise improved performance. VANETs pose many challenges for ad-hoc networks e.g. network density ($cars/m^2$) varies a lot during the day. Additionally, dynamics of the network due to the speed and the movement of cars on the streets are very challenging. New approaches like geocasting or data dissemination protocols are proposed as alternatives which fit better to the problem domain.

In data dissemination we distinguish between data sources and sinks which are often physical nodes within the network. Besides reliability, efficiency of data transfer is another important requirement for data dissemination protocols. In the case of several sinks, data transfer needs to be organized in an efficient way. In other scenarios several sources send data to one or many sinks. Here, data aggregation which means "combining" data from several sources is a key issue which is named in-network processing in WSNs. In the context of this problem we address the issue how we can build applications using such data dissemination protocols. As ad-hoc networks do not guarantee permanent connectivity the protocols need to be also delay tolerant.

Today, descriptions of algorithms and frame formats for these protocols exist but there is no common API describing how to use them. Imagine you had a complete description of the TCP or UDP Protocol but no implementation and no common Socket-API. Without API and reference implementations the TCP/IP success story might never have happened.

Therefore, the contributions of this paper are twofold. First, we propose a general purpose API for data dissemination protocols. To the best of our knowledge this is the first paper handling this issue. Second, we provide a complete evaluation of our API and AutoCast in two application domains namely WSNs and VANETs. In wireless sensor networks we decided to use time synchronization as an application as this is a common problem in sensor networks and applications may benefit from synchronized clocks. As second example we implemented a VANET application which detects and reports traffic congestions.

The rest of the paper is organized as follows: In Section II we discuss related work of data dissemination with focus on building applications and APIs for applications using data dissemination. Section III provides basics of AutoCast and some extensions which are necessary to provide a well-defined service interface to the applications. Section IV is the main part of the paper describing the API. In two exemplary applications in Section V and Section VI we evaluate our approach. The paper will conclude with a summary and a future outlook.

II. Related Work

In this section we present related work for data dissemination with focus on APIs in the two problem domains.

Reliable dissemination of data in static and mobile ad-hoc networks is tackled in several projects in the past. In the field of Delay Tolerant Networks (DTN), a protocol named OPF (Optimal Probabilistic Forwarding) is proposed in [4]. It aims to maximize the delivery probability based on specific knowledge of the underlying network and is designed to work efficiently in natural human-related mobile networks and can be extended to increase efficiency. However, the extensions are directly incorporated into OPF. The authors do not propose an API defining how additional extensions or improvements can be added to enhance the dissemination process and how OPF is using applications collaborate. In contrast to AutoCast and its API proposed in this paper, the protocol OPF is not designed with generality in mind and with the goal to support arbitrary applications.

The social-based forwarding algorithm BUBBLE Rap [5] aims to increase the delivery performance of information in Pocket Switched Networks (PSNs), a type of DTN. Unlike OPF, it does not utilize routing information but social metrics as they are intrinsic properties of PSNs. Using real human mobility traces, the approach exploits two social metrics, namely community and centrality to enhance data forwarding in PSNs. However, BUBBLE Rap is designed for a restricted set of networks and no API is provided. The authors neither show how applications can use the protocol nor how to extend its functionality and performance.

In recent years, VANETs [3] have gained importance both in academia and industry. Applications applied in these highly dynamic networks require communication protocols which smoothly adapt to the dynamic environment and participants.

In [6] the authors introduce *eSBR*, a warning message dissemination scheme which aims to mitigate the broadcast storm problem in real urban scenarios. Real city maps are evaluated on-line to sooth interferences introduced by mapped obstacles along the roads. However, no API is provided and it is no general purpose data dissemination approach since it bases on up-to-date city maps.

A distributed VANET application is introduced in [7] which detects the number of free parking spaces. A data aggregation technique is proposed which allows for continuous aggregation to create higher level aggregates. This approach does not focus on the dissemination of data using a suitable API but as we will show in Section VI, our proposed API enables disseminating data aggregates.

III. Data Dissemination with AutoCast

Transport protocols like AutoCast transfer or distribute data in the network. AutoCast has been designed as a general purpose dissemination protocol inspired by human gossiping where news spread by exchange of messages fast and efficiently. AutoCast is suitable for distribution of arbitrary data in static and mobile ad-hoc networks and can be applied by a multiplicity of distributed applications. Since ad-hoc networks do not guarantee direct communication between two arbitrary nodes multi-hop communication or piggybacking of data is required. For the rest of this paper we introduce the term data unit independent of data types used in the application if we mean data that needs to be disseminated in the network. A dissemination protocol delivers data to all sinks in an efficient way: as fast as possible but at a preferably low data rate. The payload in data units comprises destination and eventually lifetime of the data. Like in all multi-hop scenarios, nodes forward even if the actual data is not of interest to them.

To disseminate data units within network partitions, AutoCast uses adaptive probabilistic flooding and recovery mechanisms. Each node individually decides whether to forward a data unit based on locally measured parameters. The more neighbors the lesser the probability that a node will take part in the dissemination process. The number of neighbors is measured by periodic beacon messages that adapts automatically to the speed of the nodes. AutoCast adapts seamlessly to varying network densities and switches from probabilistic flooding to recovery mechanisms to bridge network partitions: It piggybacks data units and restarts forwarding as soon as new nodes from other partitions move into communication range.

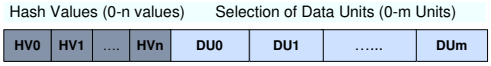

Figure 1. Basic Message Format of AutoCast with Hash Values (HV) and Data Units (DU)

AutoCast employs a single message format for beacons and for data dissemination as illustrated in Figure 1. Hash values identify data units and reduce message size. Each node maintains a list comprising hash values of known data units which is attached to each message originated by a node (cf. Figure 1) for efficient data dissemination. When node *A* receives a message from node *B*, *A* compares the received list of hash values to its own to detect data units which are mutually unknown. *A* will handle the message as implicit request and sends a message containing all available and `ACTIVE` data units missing at *B*. This broadcast message which also comprises *A*'s list of hash is an implicit request in turn too. Afterwards, *A* will schedule an explicit request for all data units which are known to *B* but not to itself.

We distinguish the following dissemination actions leading to AutoCast messages in decreasing priority:

1) Answer: A node broadcasts requested data units.
2) Flooding: Nodes broadcast new created or received data units.
3) Request: Missing data units are explicitly requested.

All actions are triggered by timers set up with increasing delays. If a timer expires, AutoCast checks whether actions can be combined (e.g. answer and flooding in one step). Scheduled explicit requests are aborted if missing data units were received in the meantime. This could be due to a neighbor answering an implicit request, like *A* does in the example above. A major strength of AutoCast is that it adapts well to changing condition. For more details of AutoCast we refer to [8].

To enable modularization and reuse as a preparation for the API we introduce here in this work states to data units which are inspired by human interaction. We suggest this as an extension to all data dissemination protocols:

1) `ACTIVE`: Up-to-date information is "interesting" in current neighborhood and is stored and disseminated.
2) `STALE`: Outdated or displaced information is stored but not disseminated.
3) `INVALID`: Invalid information is simply discarded.
4) `UNKNOWABLE`: Unknowable information is not disseminated but its hash value is stored for a while to prevent implicit requests.

In the previous version of AutoCast, the states and hash values were computed by AutoCast itself. We will show in the next section how a data unit's state and hash value are computed and used by applications using our proposed API.

IV. API for Data Dissemination

In this section, we describe a general purpose API for data dissemination protocols and illustrate our design using the example of AutoCast.

As introduced in the previous section, data units are processed by the dissemination protocol based on states and hash values. This enables applications to implement domain specific data and processing by providing appropriate states and hash value. Thereby, AutoCast supports applications inspired by nature and biology in particular as they can define appropriate actions according to the semantics of data.

We present our API with two supporting illustrations: The architecture in Figure 2 and a typical sequence of API usage in Figure 3. One instance of AutoCast is running on every device serving one or several applications. Applications are instances listing for arrival of new data and register themselves previously by invoking addDataUnitListener(). Applications call send() of AutoCast to transmit new data units and getDataUnits() to retrieve already stored ones. On reception of a new data unit, AutoCast passes the data unit to all `DataUnitListeners` by invoking add(). Applications return the actual state of data units. To maintain its pool of stored data units, AutoCast periodically queries the state of already stored data units by calling getState(). As adhering a state to a data unit is a key concept of our approach, these listeners are a key component of our API.

Figure 3 illustrates an application which (a) registers at AutoCast, (b) gets a new data unit and (c) returns its state.

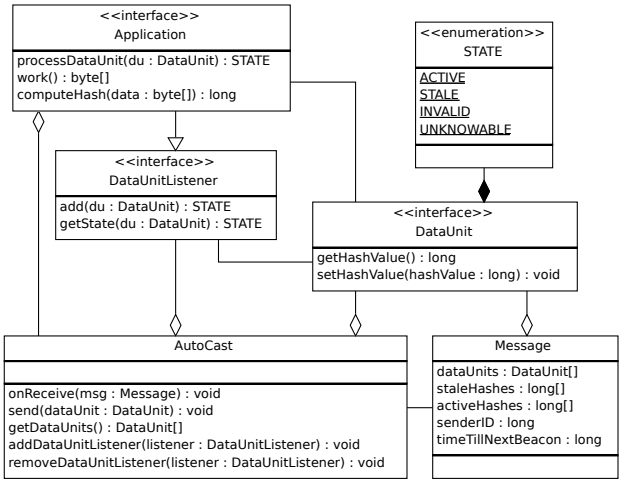

Figure 2. Simplified Class Diagram with Key Components AutoCast, DataUnit, DataUnitListener and Application

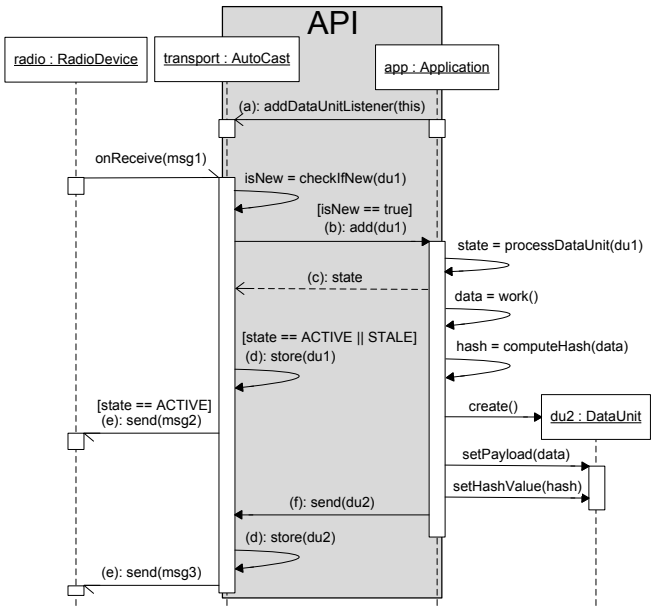

Figure 3. Reception and Dissemination of Messages with our API

AutoCast (d) stores data units on behalf of the application and (e) disseminates data units (f) provided by the application or approved for dissemination according to its state. Additionally, AutoCast queries `DataUnitListeners` for states of (h) stored data units. In Figure 3 we omitted action (h) for space restrictions. Messages `msg1` and `msg2` contain `du1`, `msg3` contains `du2`.

AutoCast discards received data units with same hash values. Therefore, hash values need to be designed carefully. Consider a data source which periodically transmits sensed values which rarely change over time. The hash value

identifying data units have to change for each update even if the sensed value has not. Otherwise, the update would not be transferred to and processed by the addressed data sinks. Since applications create hash values they can control this behavior in a flexible way.

Additionally, applications compute and return the states of data units under control (cf. Section III). `UNKNOWABLE` will be reported if an application cannot determine a data unit's state, e.g., because it cannot understand or decode it properly. `INVALID` is appropriate if the data is not valid, e.g., because its lifetime is exceeded. `STALE` will be reported if the data is valid and the packet is to be stored by the data dissemination protocol. In contrast to `ACTIVE` data units, `STALE` ones will not be disseminated in addition to these actions e.g., because they are out of their dissemination area.

With the states applications autonomously decide whether to store, transmit or discard data units. Based on this intuitive interface applications inspired by nature and biology can be built easily. The applications incorporate specific knowledge to increase the dissemination efficiency. This way, application and transport layer functionality is separated without functional dependencies between them.

In the next two sections we demonstrate the flexibility of the dissemination API by implementing applications and evaluate them in two application domains: WSNs and VANETs.

V. Evaluation Example Time Synchronization in Sensor Networks

In wireless sensor networks time synchronization is an important issue. It is needed to correlate events and synchronize medium access and sleep phases for sensor nodes. In this section we decide to use Lamport timestamps [9] to evaluate AutoCast and the API. We synchronize the clocks of sensor nodes in our testbed which consists of six nodes which are deployed in five different rooms on one floor (cf. Figure 4) and communicate via IEEE 802.15.4. They are connected to PCs for automatic programming and debugging via RS232. The setup implements the WISEBED approach [10].

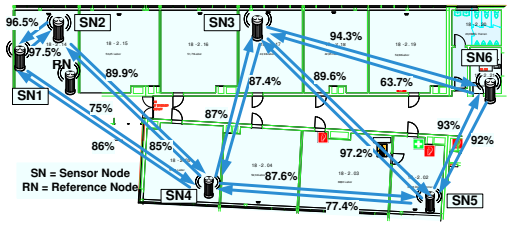

Figure 4. Testbed: Lines mark wireless Links between Sensor Nodes

The link qualities in Figure 4 are presented in percentage of successfully received packets out of 500 sent packets. None of the links reaches 100% and especially the link SN_3–SN_6 is fairly poor with many packets lost. Furthermore, the two network partitions (SN_1, SN_2 and SN_3, SN_5, SN_6) are connected by a single node SN_4 with moderate link quality. Although the network is small we need up to three hops between nodes e.g. SN_1 to SN_6.

The time synchronization application creates timestamps to be disseminated in the network. If the timestamp of a received message is larger a node's local logical time will be set to the newly received timestamp.

Measurements of the nodes' clocks show a maximum deviation of the clocks of 2% as a result of the MCU-clock deviation as illustrated in Figure 5. In the time interval from 100s to 105s the clock between the fastest (SN_4) and the slowest clock (SN_6) is 100ms which results in approximately 2%. The reason for this large deviation is that the clock is created by the internal RC-Oscillator of the microcontroller which results in a large variation between nodes. At 105s synchronization of clocks with Lamport timestamp is shown where the clock of SN_6 and SN_5 are adjusted to the new value.

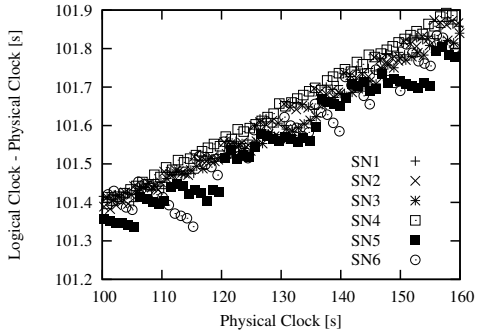

Figure 5. Synchronization of Clocks with Lamport-Algorithm without AutoCast

We implemented two versions of this algorithm with the goal to reach an accuracy of 100ms in the whole network. In the first version timestamps are exchanged between neighboring nodes exclusively each 5s. In the second version we use AutoCast to disseminate timestamps across the network. Source nodes create timestamps the same way as in the first version periodically each 5s. All timestamp data units have network wide unique hash value created as a combination of node id and a sequence number. Nodes return the state `ACTIVE` if the timestamp in the received data unit is newer than the logical clock and `STALE` if the timestamp is older as illustrated in Section III and Section IV. Consequently, AutoCast will not disseminate data units marked with `STALE`. If we compare the time synchronization with a scenario where humans adjust their clocks only new time values are propagated to neighbors as this information is more important.

The accuracy of time synchronization is evaluated in long term measurements and results are presented in Figure 7

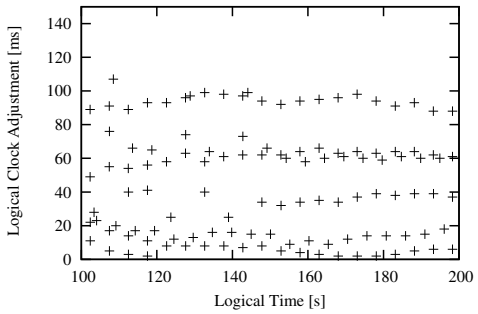

Figure 6. Accuracy without AutoCast – Sending Timestamp every 5s

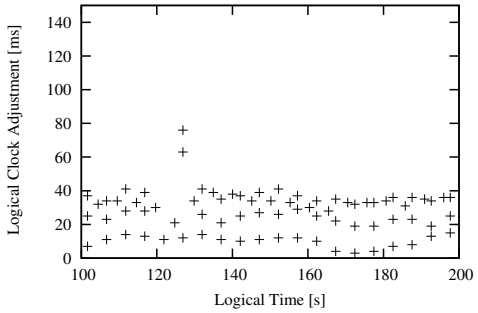

Figure 7. Accuracy with AutoCast – Sending Timestamp every 5s

and Figure 6. Compared to Figure 5 we illustrate a longer time period and only the adjustment of the clocks is been shown. Here, the adjustment serves a measurement of the accuracy of the clocks. AutoCast reaches a better accuracy (40ms) compared to the simple version (100ms) without AutoCast. During the experiment we count the number of the received timestamp messages for all nodes to measure the overhead of AutoCast. 370 timestamps are received using AutoCast and 230 timestamps without using AutoCast in a time interval of 100s. To reach the same accuracy of 40 ms like in Figure 7 without AutoCast we have to send timestamps with higher frequency, that means sending timestamps every 2 s resulting in an increase of 230 to more than 550 $(230 * 5/2)$ messages. AutoCast reaches the same result with less overhead. This example demonstrates how easy it is to implement an efficient time synchronization algorithm with the suggested API.

VI. Evaluation Example Traffic Information System

In this section we evaluate our approach in a complete different domain and show its applicability in a traffic application. We consider an application of AutoCast in a VANET scenario which features high node mobility whereat the kind of node movement as well as the velocities depend on the underlying road network [11]. On highways, nodes usually move on lanes either in the same or the opposite direction while in cities—especially with high junction densities—nodes typically move more slowly but in all directions. Due to buildings and other obstacles, radio transmission range and error rate is less predictable in urban areas [6]. Moreover, node movement and network density is time-dependent in certain areas [12].

These requirements necessitate a transport protocol, which quickly adapts to changing neighborhood and readjusts its dissemination strategy on changing network densities. AutoCast features both characteristics. To enhance the performance of AutoCast, additional application specific knowledge can be exploited, too.

Figure 8. Detection and Localization of a Traffic Accident

We implemented an application which detects and reports traffic jams based on local rules and distributed algorithms as follows: The application will monitor its hosting vehicle's velocity and transmit messages to its neighbors if it slows down unexpectedly. This could occur due to a traffic accident as depicted in Figure 8 where cars which are behind the vehicle which broke down disseminate deceleration event. Such a message comprises the detected event, the vehicle's position and a hash value which identifies the information. If a certain amount of applications running on neighboring nodes detect and report the same data, a traffic jam is detected. In this case, the application will aggregate the information and will report the detected jam. This behavior is inspired by human beings sharing information and collaboratively upgrading their knowledge about hazards, etc. in their regions of interest. More details are available in [13].

In the following we will discuss one interesting aspect of the API the computation of hash values in more detail. When reporting a traffic jam, there is no need to include the vehicle's position and speed in every detail. If a vehicle takes part in a traffic jam, a nearby neighbor will most certainly do the same. Thus, when using a coarse grained position, neighboring vehicles will create data units which are identified by the same hash value which saves communication bandwidth (cf. Section III). Our approach supports this flexibility with the concept of hash values.

Another important aspect of the application is the control of data dissemination. To enable oncoming vehicles to choose a different route, the traffic jam reports have to get to the street's previous junction or even farther. Therefore, applications will use a dissemination area layout based on the road network layout or, e.g., a circular shaped area based on absolute coordinates if no accurate maps are available.

However, a general purpose data dissemination protocol cannot provide custom-tailored dissemination areas for each and every application. Thus, it is beneficial to implement the dissemination model in the application layer and let applications implement their own dissemination strategies. The states of data units are the interface (handle) that is available to the application to control dissemination efficiently.

Hence, a data dissemination protocol attached to an application by our API queries the application for approval before transmitting and storing data units. This way, applications can directly control where and for how long their data is disseminated while the data dissemination protocol makes sure that the actual dissemination strategy adapts to changing node densities and network topologies.

VII. Conclusion and Future Work

In this paper we suggest an API which allows applications to use a general purpose dissemination protocol like AutoCast. The approach provides modularization, encapsulation & separation of functionalities. The dissemination protocol and the application are well separated with clear interface which allows for reuse of applications and the underlying dissemination protocol.

We have presented a summary of AutoCast as one implementation of data dissemination protocol inspired by nature and refer to [8] for a complete description of the algorithm and evaluation. We have demonstrated how we can build applications that control the dissemination process by returning hashes and states for data units using our API. We believe that these key components support applications and protocols that are inspired by nature and biology. While the decision about storing or disseminating data units is done at application layer, dissemination protocols autonomously decide about dissemination strategy, e.g., to anticipate broadcast storms and to bridge network partitions.

We evaluated the approach by using two example implementations with AutoCast in static and mobile ad-hoc networks. In both fields our API provides flexibility to build challenging applications which efficiently disseminate data using AutoCast. We foresee a large potential by this separation of transport and application layer for data dissemination which provides us with a large degree of flexibility for hovering data through a network including dissemination direction and also speed at application layer.

In the future we will investigate the API in more detail and show how to address memory limitations. We will further investigate the energy consumption of our approach where we expect to trade-off energy versus accuracy in the application layer with different states. We can control energy consumption efficiently as data transmission in WSNs is by far more energy consuming than local computations.

Acknowledgments

The authors thank the DFG and the Federal Ministry of Education & Research Germany for funding the projects AutoNomos within the Priority Programme "Organic Computing" (SPP 1183) and DataCast (ID 17PNT017).

References

[1] M. Fiore, J. Harri, F. Filali, and C. Bonnet, "Vehicular mobility simulation for vanets," in *Proceedings of the 40th Annual Simulation Symposium*. Washington, DC, USA: IEEE Computer Society, 2007, pp. 301–309.

[2] K. Bilstrup, E. Uhlemann, E. G. Strom, and U. Bilstrup, "Evaluation of the IEEE 802.11p MAC Method for Vehicle-to-Vehicle Communication," in *Vehicular Technology Conference, 2008. VTC 2008-Fall. IEEE 68th*, 2008, pp. 1–5.

[3] T. Kosch, C. J. Adler, S. Eichler, C. Schroth, and M. Strassberger, "The scalability problem of vehicular ad hoc networks and how to solve it," *IEEE Wireless Communications*, vol. 13, Number: 5, pp. 22 – 28, 2006.

[4] C. Liu and J. Wu, "An optimal probabilistic forwarding protocol in delay tolerant networks," in *the tenth ACM international symposium*. New York, New York, USA: ACM Press, 2009.

[5] P. Hui, J. Crowcroft, and E. Yoneki, "Bubble rap: Social-based forwarding in delay tolerant networks," *IEEE Transactions on Mobile Computing*, vol. 99, no. PrePrints, 2010.

[6] F. J. Martinez, M. Fogue, M. Coll, J.-C. Cano, C. M. T. Calafate, and P. Manzoni, "Evaluating the impact of a novel warning message dissemination scheme for vanets using real city maps," in *Networking*, ser. Lecture Notes in Computer Science, M. Crovella, L. M. Feeney, D. Rubenstein, and S. V. Raghavan, Eds., vol. 6091. Springer, 2010, pp. 265–276.

[7] C. Lochert, B. Scheuermann, and M. Mauve, "A probabilistic method for cooperative hierarchical aggregation of data in vanets," *Ad Hoc Networks*, vol. 8, no. 5, pp. 518–530, 2010.

[8] H. Hellbrück, A. Wegener, and S. Fischer, "Autocast: A general-purpose data dissemination protocol and its application in vehicular networks," *Ad Hoc & Sensor Wireless Networks Journal (AHSWN)*, pp. 91–22, 2008.

[9] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, July 1978.

[10] H. Hellbrck, M. Pagel, A. Krller, D. Bimschas, D. Pfisterer, and S. Fischer, "Using and operating wireless sensor network testbeds with wisebed," in *Proceedings of the 10th IEEE IFIP Annual Mediterranean Ad Hoc Networking Workshop*, Favignana Island, Sicily, Italy, Jun. 2011.

[11] M. Jerbi, S.-M. Senouci, R. Meraihi, and Y. Ghamri-Doudane, "An improved vehicular ad hoc routing protocol for city environments," in *ICC*. IEEE, 2007, pp. 3972–3979.

[12] A. Downs, *Still Stuck in Traffic: Coping with Peak-Hour Traffic Congestion*. Brookings Institution Press, 2004.

[13] S. P. Fekete, C. Schmidt, A. Wegener, H. Hellbrück, and S. Fischer, "Empowered by wireless communication: Distributed methods for self-organizing traffic collectives," *ACM Transactions on Autonomous and Adaptive Systems*, vol. 5, no. 3, pp. 439–462, 2010.